AWS Developer Exam: Domain 3 – Prepare Application Artifacts to Be Deployed to AWS

Overview

This task statement focuses on preparing application artifacts for deployment to AWS. Key themes include:
Artifact repositories and storage
Deployment packages for Lambda and serverless applications
Directory structure, version control, and code repositories
Managing dependencies in deployment packages
Accessing application configuration data

Artifact Repositories

Developers use artifact repositories to store and share versioned software or deployment packages. Benefits include:
Increased code reuse
Reduced delivery time
Tighter artifact governance
Improved security visibility

AWS CodeArtifact

AWS CodeArtifact is a secure, highly scalable, managed artifact repository service. It eliminates the need to manage your own artifact storage system or scale its infrastructure.

Amazon S3

S3 is another option for storing application artifacts — particularly deployment packages, zip files, and SAM-packaged templates.

AWS SAM (Serverless Application Model)

SAM is used to build and deploy serverless applications. CloudFormation serves as the underlying deployment mechanism.

Key SAM CLI Commands

Command	What It Does
`sam package`	Zips code artifacts, uploads them to S3, and produces a packaged SAM template file ready for deployment
`sam deploy`	Deploys the packaged application using the SAM/CloudFormation template
`sam build --use-container`	Builds a deployment package inside a Docker image compatible with the Lambda execution environment (useful when native libraries are involved)

SAM Template — Transform Section

When creating a CloudFormation template that includes a SAM script and other service configurations (e.g., to replicate infrastructure in another Region), you must include the `Transform` **section** in the template to specify the version of SAM being used.

The `Transform` section is what tells CloudFormation to process the template as a SAM application.

AWS Lambda Deployment Packages

Lambda supports two types of deployment packages:

1. Zip File Archives

Standard deployment method for Lambda functions.

Dependencies and libraries are bundled into the zip file or pulled in via **Lambda Layers**.

If using Node.js and the function depends on libraries beyond the AWS SDK, use **NPM** to include them in the deployment package.

If any libraries use **native code**, build the deployment package using an **Amazon Linux environment** (or use `sam build --use-container`).

You can include a specific version of the AWS SDK in your package if you need a newer version or want to pin it.

2. Container Images

Package your preferred runtime, libraries, and other dependencies into a **container image** at build time.

Lambda Layers are not used with container images — everything is bundled into the image itself.

Lambda Layers

Lambda Layers allow you to manage function dependencies **independently** from your core function code.

Layers are useful when:

Your deployment package would exceed size limits (direct upload limit applies).

You want to share libraries, custom runtimes, or other dependencies across multiple functions.

You want to separate unchanging code/resources from your active development code.

Layer Sources

You can configure a Lambda function to use:

Layers **you create**

Layers **AWS provides**

Layers from **other AWS customers**

Exam Scenario: A Lambda function exceeds 100 MB — use **Lambda Layers** to pull in additional code and dependencies from a separate source, reducing the deployment package size. (Not environment variables or aliases.)

AWS CDK (Cloud Development Kit)

CDK is an infrastructure as code (IaC) solution that allows you to define cloud resources using familiar programming languages.

Dependencies for CDK applications or libraries are managed using **package management tools** such as **PIP** (Python), **Yarn** (Node.js), **NPM**, and others.

Code Repositories and Version Control

Version control is essential for:
Storing and tracking code revision history
Merging code changes from multiple contributors
Reverting to earlier code versions when needed

Deployment with CodePipeline + CodeDeploy

A common pattern:
Code is maintained in a repository (e.g., CodeCommit).
A **CodePipeline** is triggered on each push to the repo.
The pipeline deploys changes to an EC2 instance using **AWS CodeDeploy** as the deployment service.

Applying Application Requirements in Deployments

AWS CloudFormation

CloudFormation can automatically **install, configure, and start applications on EC2 instances**. This enables:
Duplicating deployments across environments or Regions
Updating existing installations without connecting directly to instances
CloudFormation includes **helper scripts** based on `cloud-init` that you call from within templates to install, configure, and update applications on EC2 instances defined in the same template. The template itself serves as documentation of what is required to deploy the application.

AWS Systems Manager — Key Capabilities for Deployment

Feature	Purpose
Application Manager	View application resources, monitor information, and access log data
AppConfig	Create, manage, and quickly deploy application configurations to applications of any size; includes built-in validation and monitoring
Parameter Store	Hierarchical, secure storage for configuration data and secrets

AWS AppConfig

AppConfig is a Systems Manager capability that:
Creates, manages, and deploys application configurations
Supports applications hosted on EC2, Lambda, containers, mobile apps, and IoT devices
Includes **built-in validation checks** and **monitoring** to prevent bad configurations from being deployed

ECS Task Placement Strategies

When hosting an application in an ECS cluster with a requirement to minimize the number of instances in use, choose the right **task placement strategy**:

Strategy	Description
Binpack	Places tasks based on the **least available CPU and memory** — minimizes the number of instances in use
Spread	Distributes tasks evenly across instances or AZs
Distinct Instance	Places each task on a different container instance
Random	Places tasks randomly

Exam Scenario: Minimize the number of instances in use → **Binpack** is the correct placement strategy.

Managing Dependencies in Deployment Packages

Lambda (Node.js Example)

Use the **AWS SDK** already available in the Lambda execution environment, or include a specific version in the package.

Use **NPM** to include external libraries in the deployment package.

For native code libraries → build the package on **Amazon Linux** or use `sam build --use-container`.

Lambda Layers (Alternative)

Include layers to use libraries **without** bundling them in the deployment package — keeps the core function code lean.

CDK Applications

Use standard language-specific package managers:

PIP for Python

NPM / Yarn for Node.js

And others depending on the language

Accessing Application Configuration Data

Parameter Store + Secrets Manager Integration

You can use **Parameter Store to reference Secrets Manager secrets**, creating a consistent and secure process for calling secrets and reference data in your code and configuration scripts.

Services that support references to Parameter Store parameters include:

EC2, ECS, Lambda, CloudFormation, CodeBuild, CodeDeploy, and other Systems Manager capabilities

Lambda — Accessing Parameters Without the SDK

Use the **AWS Parameters and Secrets Lambda Extension** to:

Retrieve Parameter Store parameter values directly in Lambda
Cache retrieved values for future use
Avoid the overhead of SDK calls in every invocation

AWS AppConfig for Lambda and Other Compute

AppConfig can also manage and deploy application configurations to Lambda functions, containers, EC2, mobile apps, and IoT devices — without needing to redeploy code when configuration changes.

Key Exam Tips

`sam package` zips artifacts, uploads to S3, and produces the packaged template. `sam deploy` deploys it.
The `Transform` **section** in a CloudFormation template specifies the SAM version — required for SAM-based deployments.
Lambda Layers are for zip-based deployments only. With container images, all dependencies are bundled into the image at build time.
Use **Lambda Layers** when a function exceeds deployment size limits — not environment variables or aliases.
Binpack is the ECS task placement strategy for minimizing instance count.
For **on-premises or native library dependencies** in Lambda (Node.js), build on Amazon Linux or use `sam build --use-container`.
AWS CodeArtifact is the managed artifact repository service — eliminates the need to manage your own storage and scaling.
CloudFormation helper scripts (based on `cloud-init`) allow you to install, configure, and start apps on EC2 instances from within the template.
Use the **Parameters and Secrets Lambda Extension** to retrieve and cache Parameter Store values in Lambda without using the SDK.
AppConfig adds validation and monitoring to configuration deployments — use it for any compute type (Lambda, EC2, ECS, IoT, mobile). EOF